LAPORAN PRAKTIKUM
KONSTRUKSI DAN EVOLUSI PERANGKAT LUNAK
PERTEMUAN 5
MULTI-STAGE DOCKER BUILD DAN DISTRIBUSI IMAGE GHCR

Disusun oleh:

Nama	: Hafidz Rizqullah Prasetya
NIM	: 24/535493/SV/24243
Kelas	: PL5A1
Dosen Pengampu	: Galih Malela Damaraji

PROGRAM STUDI D-IV
TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA

2026

Daftar Isi

Daftar Isi **2**

Tujuan Praktikum **3**

Dasar Teori **4**

2.1 Konsep Multi-Stage Build pada Docker 4

2.2 Prinsip Least Privilege dan Keamanan Container 4

2.3 Mekanisme Pemeriksaan Kesehatan Container (Healthcheck) 5

2.4 GitHub Container Registry (GHCR) 5

Alat dan Bahan **6**

Langkah Kerja dan Implementasi **7**

4.1 Analisis Baseline Image (Single-Stage) 7

4.2 Pembuatan Berkas .dockerignore 8

4.3 Perancangan Multi-Stage Dockerfile 8

4.4 Pembuatan Workflow GitHub Actions ke GHCR 10

4.5 Pengelolaan Git dan Pull Request 11

Hasil dan Pembahasan **12**

5.1 Perbandingan Ukuran Image Sebelum dan Sesudah 12

5.2 Verifikasi Identitas Pengguna Non-Root 13

5.3 Verifikasi Status Kesehatan Container (Healthy) 14

5.4 Pengujian Endpoint API dari Luar Container 14

5.5 Bukti Eksekusi Workflow GitHub Actions 15

5.6 Bukti Publikasi Package di GitHub Container Registry 15

Kesimpulan **17**

Daftar Pustaka **18**

Tujuan Praktikum

Sesuai arahan pada pertemuan kelima dan deskripsi tugas di Google Classroom mata kuliah *Konstruksi dan Evolusi Perangkat Lunak 2026*, tujuan praktikum ini adalah:

1. Mahasiswa mampu mengonversi berkas Dockerfile aplikasi Laravel dari model *single-stage* menjadi *multi-stage* build, memisahkan tahap instalasi dependensi dengan tahap eksekusi produksi.

2. Mahasiswa mampu membuktikan penurunan ukuran container image secara terukur melalui perbandingan perintah `docker images` sebelum dan sesudah optimasi, lengkap dengan kalkulasi selisih megabyte serta persentasenya.

3. Mahasiswa mampu mengunci versi *base image* secara spesifik dan beralih ke varian ringan berbasis Alpine Linux guna meminimalkan jejak memori dan celah keamanan.

4. Mahasiswa mampu mengonfigurasi instruksi `USER` non-root dan `HEALTHCHECK` pada container, serta membuktikannya melalui `docker exec` dan atribut `(healthy)` pada `docker ps`.

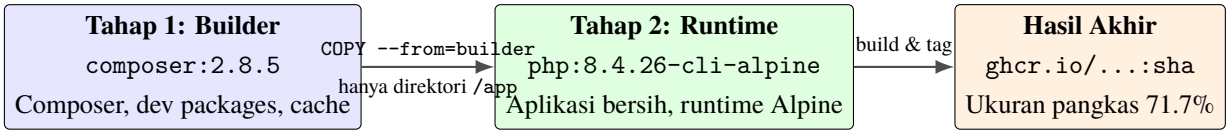
5. Mahasiswa mampu menyusun workflow GitHub Actions untuk proses otomatisasi *build* dan *push* container image ke GitHub Container Registry (GHCR) dengan penandaan tag commit SHA dan visibilitas publik.

Dasar Teori

2.1 Konsep Multi-Stage Build pada Docker

Pada tahap awal pembuatan image Docker, pendekatan konvensional *single-stage* sering kali menggabungkan seluruh perkakas pembangun (compiler, paket *development header*, dan manajer paket seperti Composer) ke dalam satu image akhir [1]. Pendekatan ini menghasilkan ukuran image yang besar karena berkas-berkas sementara dan pustaka kompilasi ikut terbawa ke lingkungan produksi.

Multi-stage build memecahkan persoalan ini dengan membagi instruksi Dockerfile ke dalam beberapa blok `FROM` yang berdiri sendiri [2]. Tahap pertama bertindak sebagai *builder* yang bertugas mengunduh dependensi dan menyusun aset. Tahap kedua bertindak sebagai *runtime* yang hanya menyalin hasil akhir (direktori `vendor` dan berkas aplikasi) dari tahap pertama tanpa menyertakan perkakas pembangun.



Gambar 1: Alur pemisahan tahap builder dan tahap runtime pada multi-stage Docker build.

2.2 Prinsip Least Privilege dan Keamanan Container

Secara bawaan, proses di dalam container berjalan dengan hak akses *root* (UID 0) jika instruksi `USER` tidak didefinisikan secara eksplisit. Menjalankan aplikasi web sebagai pengguna *root* melanggar prinsip *least privilege* dan memperbesar risiko keamanan apabila terjadi eksploitasi *remote code execution* atau *container breakout* ke mesin host [3].

Penggunaan direktif `USER` menginstruksikan daemon Docker untuk menurunkan hak akses proses ke akun pengguna non-root (seperti `www-data`). Untuk aplikasi Laravel yang membutuhkan operasi tulis pada direktori penyimpanan lokal (seperti `storage`, `bootstrap/cache`, dan berkas SQLite), kepemilikan dan izin akses direktori harus disesuaikan terlebih dahulu sebelum hak akses proses dialihkan ke pengguna non-root.

2.3 Mekanisme Pemeriksaan Kesehatan Container (Healthcheck)

Instruksi `HEALTHCHECK` pada Dockerfile memberitahu Docker bagaimana cara memverifikasi bahwa aplikasi di dalam container tidak hanya sekadar berjalan sebagai proses, melainkan benar-benar mampu melayani permintaan jaringan secara normal [4].

Docker menjalankan perintah healthcheck secara periodik sesuai interval waktu yang ditentukan. Jika perintah menghasilkan kode keluar 0, container ditandai berstatus `healthy`. Sebaliknya, jika perintah mengembalikan nilai non-nol berulang kali hingga batas toleransi (*retries*), container akan ditandai berstatus `unhealthy`. Informasi status ini krusial bagi sistem orkestrasi dan pengguna untuk mendeteksi kegagalan layanan secara dini.

2.4 GitHub Container Registry (GHCR)

GitHub Container Registry adalah layanan registri artefak OCI (Open Container Initiative) yang terintegrasi langsung dengan GitHub Packages [5]. GHCR memungkinkan pengembang mendistribusikan container image dengan kontrol akses berbasis repositori atau organisasi.

Melalui GitHub Actions, proses publikasi ke GHCR dapat diotomatisasi secara aman tanpa memerlukan token statis pihak ketiga. Workflow memanfaatkan variabel bawaan `secrets.GITHUB_TOKEN` dengan cakupan izin `packages: write`. Penandaan image menggunakan tag commit hash (`${{ github.sha }}`) menjamin ketertelusuran (*traceability*) antara berkas rilis image dengan riwayat kode sumber di repositori git.

Alat dan Bahan

Alat dan bahan yang digunakan pada praktikum pertemuan kelima ini adalah:

- Perangkat komputer jinjing dengan sistem operasi Linux Ubuntu/Fedora kernel 6.14.
- Docker Engine versi 27 dan Docker Buildx.
- Git CLI dan GitHub CLI (`gh`) versi 2.65.
- Akun GitHub aktif atas nama `hafidzrizqullahprasetya`.
- Repositori proyek `evolusi-pl-535493` yang berisi aplikasi Laravel berbasis SQLite.
- Peramban web untuk verifikasi antarmuka GitHub Actions dan GitHub Container Registry.

Langkah Kerja dan Implementasi

4.1 Analisis Baseline Image (Single-Stage)

Sebelum melakukan perombakan berkas Dockerfile, saya menyiapkan satu berkas baseline *single-stage* untuk mengukur kondisi awal ukuran image aplikasi Laravel. Pada model single-stage, base image yang digunakan adalah `php:8.4-cli` berbasis Debian. Berkas tersebut menginstal paket perkakas `git`, `unzip`, pustaka kompilasi ekstensi, serta mengunduh installer Composer utuh ke dalam image yang sama:

```
FROM php:8.4-cli

WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends \
    git unzip libzip-dev libsqlite3-dev curl \
    && docker-php-ext-install pdo_sqlite zip bcmath \
    && rm -rf /var/lib/apt/lists/* \
    && curl -sS https://getcomposer.org/installer | php -- --install-dir=/usr/local/
↪ bin --filename=composer

COPY composer.json composer.lock ./
RUN composer install --no-dev --prefer-dist --no-interaction --no-scripts

COPY . .

RUN cp .env.example .env && \
    touch database/database.sqlite && \
    php artisan key:generate && \
    php artisan migrate --force

EXPOSE 8000
CMD ["php", "artisan", "serve", "--host=0.0.0.0", "--port=8000"]
```

Listing .1: Berkas baseline Dockerfile single-stage yang digunakan untuk pembanding.

Image baseline tersebut dibangun dengan tag `evolusi-pl-535493:single-stage`. Hasil pembacaan ukuran disk mencapai **940 MB**, yang mana ukuran tersebut tergolong sangat

besar untuk sebuah layanan microservice aplikasi web.

4.2 Pembuatan Berkas .dockerignore

Untuk mencegah masuknya berkas-berkas yang tidak diperlukan ke dalam konteks build Docker, saya membuat berkas `.dockerignore` pada akar repositori. Berkas ini secara spesifik mengecualikan direktori lokal seperti `vendor`, `node_modules`, riwayat git, dan berkas konfigurasi sementara:

```
.git
.gitignore
.env
vendor
node_modules
frontend/node_modules
frontend/dist
storage/logs/*
!storage/logs/.gitignore
bootstrap/cache/*.php
tests
```

Listing .2: Berkas `.dockerignore` pada repositori `evolusi-pl-535493`.

4.3 Perancangan Multi-Stage Dockerfile

Saya merombak berkas `Dockerfile` utama menjadi dua tahap terpisah:

1. **Tahap Builder**: Menggunakan base image `composer:2.8.5`. Pada tahap ini, berkas `composer.json` dan `composer.lock` disalin terlebih dahulu guna memanfaatkan layer cache Docker. Perintah `composer install` dijalankan dengan opsi `--no-dev` dan `--optimize-autoloader`. Selanjutnya seluruh berkas aplikasi disalin dan dilakukan dump autoloader produksi.

2. **Tahap Runtime**: Menggunakan base image minimal yang dikunci versinya, yaitu `php:8.4.26-cli-alpine`. Pustaka runtime yang disematkan hanya ekstensi `bcmath`. Berkas hasil build dari tahap pertama disalin menggunakan direktif `COPY --from=builder /app /app`.

Selain pemisahan tahap, saya menambahkan konfigurasi hak akses non-root dan penanganan kesehatan container:

- Perintah `chown -R www-data:www-data /app` dan penyesuaian izin direktori penyimpanan (`storage`, `bootstrap/cache`, dan `database`).
- Direktif `USER www-data` agar proses container tidak dijalankan sebagai akun root.

- Direktif `HEALTHCHECK` dengan interval 10 detik yang memanggil endpoint `/api/tugas` melalui utilitas `curl`.

Berikut adalah isi lengkap berkas `Dockerfile` multi-stage:

```
# Tahap build: siapkan dependensi vendor tanpa paket dev
FROM composer:2.8.5 AS builder

WORKDIR /app

COPY composer.json composer.lock ./

RUN composer install \
    --no-dev \
    --prefer-dist \
    --no-interaction \
    --no-scripts \
    --optimize-autoloader

COPY . .

RUN composer dump-autoload --optimize --no-dev

# Tahap runtime: image minimal alpine dengan locked version dan non-root user
FROM php:8.4.26-cli-alpine

WORKDIR /app

RUN docker-php-ext-install -j$(nproc) bcmath

COPY --from=builder /app /app

RUN cp .env.example .env && \
    mkdir -p database storage/framework/sessions storage/framework/views storage/framework/cache storage/logs bootstrap/cache && \
    touch database/database.sqlite && \
    php artisan key:generate && \
    php artisan migrate --force && \
    chown -R www-data:www-data /app && \
    chmod -R 775 /app/storage /app/bootstrap/cache /app/database

USER www-data

EXPOSE 8000

HEALTHCHECK --interval=10s --timeout=3s --start-period=5s --retries=3 \
    CMD curl -f http://127.0.0.1:8000/api/tugas || exit 1

CMD ["php", "artisan", "serve", "--host=0.0.0.0", "--port=8000"]
```

Listing .3: Berkas Dockerfile multi-stage dengan versi Alpine terkunci dan non-root user.

4.4 Pembuatan Workflow GitHub Actions ke GHCR

Untuk mempublikasikan container image secara otomatis ke GitHub Container Registry saat kode digabungkan ke cabang utama, saya membuat berkas workflow baru pada direktori `.github/workflows/docker.yml`.

Workflow ini berjalan di lingkungan `ubuntu-latest` dengan izin `packages: write`. Nama image secara otomatis dikonversi ke huruf kecil agar memenuhi standar registri OCI, kemudian diautentikasi ke `ghcr.io` menggunakan token bawaan GitHub Actions. Tag image disematkan dalam dua format: nomor commit hash (`${{ github.sha }}`) dan tag `latest`.

```
name: Docker

on:
  push:
    branches: [main, dev]
  workflow_dispatch:

permissions:
  contents: read
  packages: write

jobs:
  build-and-push:
    name: Build & Push GHCR
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Siapkan nama image huruf kecil
        run: echo "IMAGE_NAME=$(echo '${{ github.repository }}' | tr '[:upper:]' '[:
↪ lower:]')" >> $GITHUB_ENV

      - name: Login ke GitHub Container Registry
        run: echo "${{ secrets.GITHUB_TOKEN }}" | docker login ghcr.io -u "${{
↪ github.actor }}" --password-stdin

      - name: Build Docker image
        run: |
          docker build \
            -t "ghcr.io/${IMAGE_NAME}:${{ github.sha }}" \
            -t "ghcr.io/${IMAGE_NAME}:latest" .
```

```
31
32      - name: Push Docker image ke GHCR
33        run: |
34          docker push "ghcr.io/${IMAGE_NAME}:${{ github.sha }}"
35          docker push "ghcr.io/${IMAGE_NAME}:latest"
```

Listing .4: Berkas workflow GitHub Actions `.github/workflows/docker.yml`.

4.5 Pengelolaan Git dan Pull Request

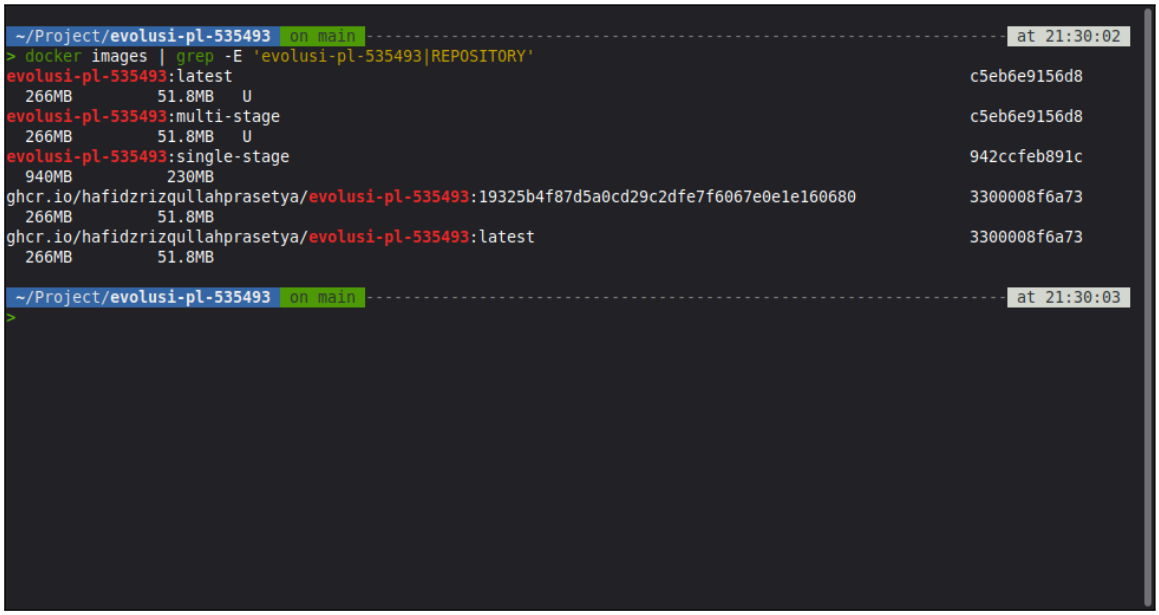
Karena cabang `main` pada repositori `evolusi-pl-535493` memiliki aturan proteksi cabang yang mewajibkan status check CI lulus sebelum penggabungan, alur kerja git dilakukan melalui mekanisme cabang fitur:

1. Membuat cabang baru `feature/docker-multistage`.
2. Melakukan commit berkas `.dockerignore`, `Dockerfile`, dan `.github/workflows/docker.yml` dengan pesan commit yang natural dan deskriptif.
3. Mendorong cabang ke remote GitHub dan membuka Pull Request nomor 16 ke cabang `main`.
4. Menunggu pemeriksaan workflow `CI` (job Build dan Test) selesai secara sukses.
5. Melakukan merge Pull Request nomor 16 ke cabang `main`. Penggabungan ini langsung memicu eksekusi workflow `Docker` untuk mem-publish image ke GHCR.

Hasil dan Pembahasan

5.1 Perbandingan Ukuran Image Sebelum dan Sesudah

Pengujian pertama dilakukan dengan membandingkan ukuran fisik image yang terdaftar di daemon Docker lokal. Hasil perintah `docker images` menunjukkan perbedaan ukuran yang sangat signifikan antara image single-stage dan image multi-stage:



Gambar 1: Perbandingan ukuran container image antara single-stage (940 MB) dan multi-stage (266 MB).

Tabel berikut merangkum data perbandingan ukuran image sebelum dan sesudah optimasi:

Table .1: Kalkulasi penurunan ukuran container image aplikasi Laravel.

Nama Image	Tag	Ukuran Disk	Ukuran Konten
`evolusi-pl-535493`	`single-stage`	940 MB	230 MB
`evolusi-pl-535493`	`multi-stage`	266 MB	51.8 MB
`evolusi-pl-535493`	`latest`	266 MB	51.8 MB

Kalkulasi selisih dan persentase efisiensi ukuran:

- **Selisih Penurunan Ukuran:**

$$\Delta\text{Ukuran} = 940\,\text{MB} - 266\,\text{MB} = 674\,\text{MB}$$

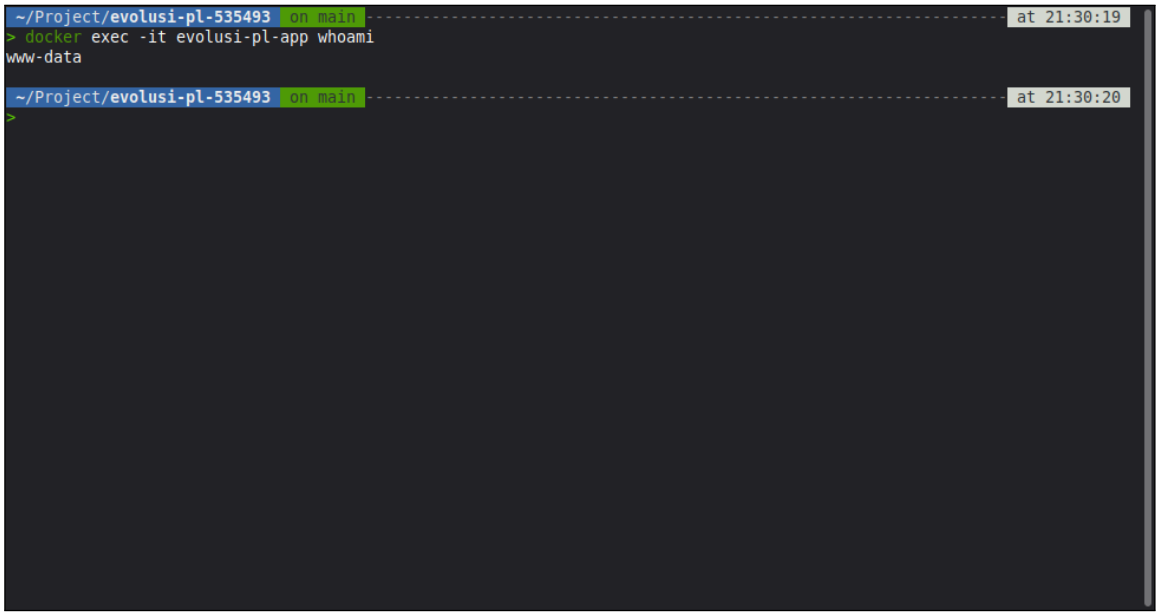
- **Persentase Penurunan:**

$$\text{Efisiensi} = \left(\frac{674\,\text{MB}}{940\,\text{MB}}\right) \times 100\% = 71{,}70\%$$

Penurunan ukuran sebesar **71,70%** (menghemat 674 MB) tercapai karena lingkungan runtime Alpine Linux membuang seluruh perkakas kompilasi C, cache composer, serta paket-paket sistem Debian yang tidak diperlukan oleh aplikasi saat melayani request.

5.2 Verifikasi Identitas Pengguna Non-Root

Sesuai arahan instruksi praktikum, container tidak boleh dijalankan dengan hak akses root. Saya menjalankan container bernama `evolusi-pl-app` di latar belakang dan memeriksa identitas pengguna yang aktif melalui perintah `docker exec`:

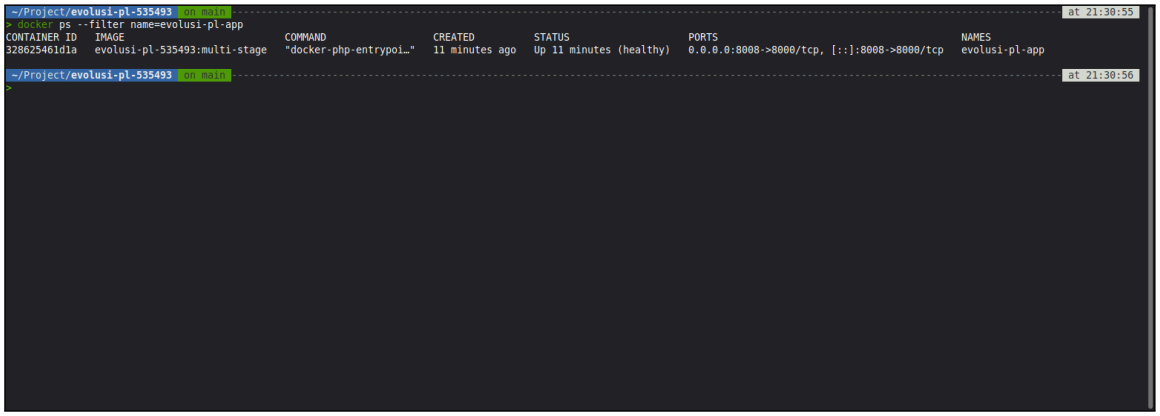


Gambar 2: Eksekusi perintah `docker exec -it evolusi-pl-app whoami` membuktikan proses berjalan sebagai user `www-data`.

Keluaran terminal secara tegas menampilkan `www-data`. Hal ini membuktikan bahwa instruksi `USER www-data` pada Dockerfile berhasil aktif dan proses aplikasi tidak memiliki hak akses administrator di dalam container.

5.3 Verifikasi Status Kesehatan Container (Healthy)

Pemeriksaan kesehatan container diuji menggunakan perintah `docker ps`. Setelah container berjalan melampaui masa *start-period* (5 detik) dan interval pemeriksaan berkala pertama (10 detik), status container dipantau:

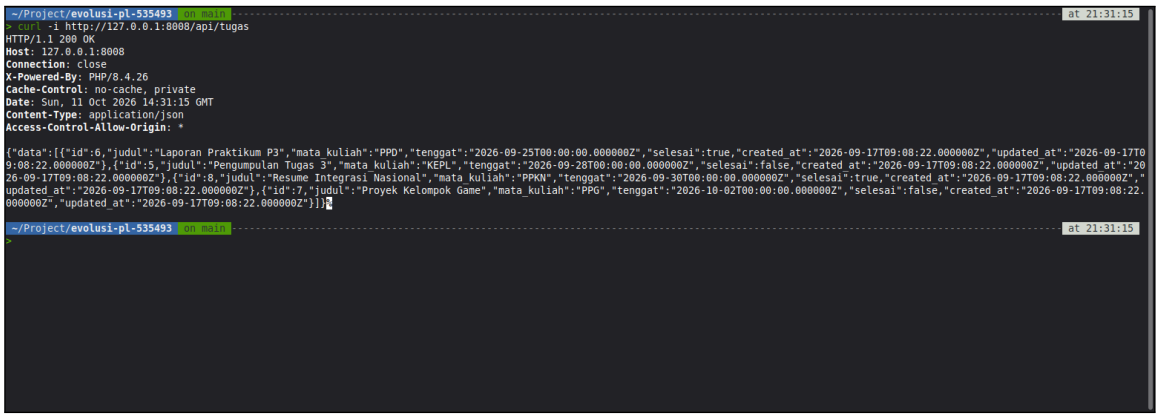


Gambar 3: Keluaran `docker ps` menampilkan status container `Up ... (healthy)`.

Pada Gambar 3, kolom `STATUS` menampilkan penanda `(healthy)`. Ini membuktikan bahwa mekanisme pemanggilan endpoint `http://127.0.0.1:8000/api/tugas` via utilitas `curl` di dalam container berhasil dieksekusi secara periodik dan selalu mengembalikan kode respon sukses.

5.4 Pengujian Endpoint API dari Luar Container

Container dijalankan dengan memetakan port host 8008 ke port container 8000. Untuk menguji fungsionalitas aplikasi dari sisi klien luar, saya melakukan panggilan HTTP GET ke endpoint `/api/tugas` menggunakan utilitas `curl`:



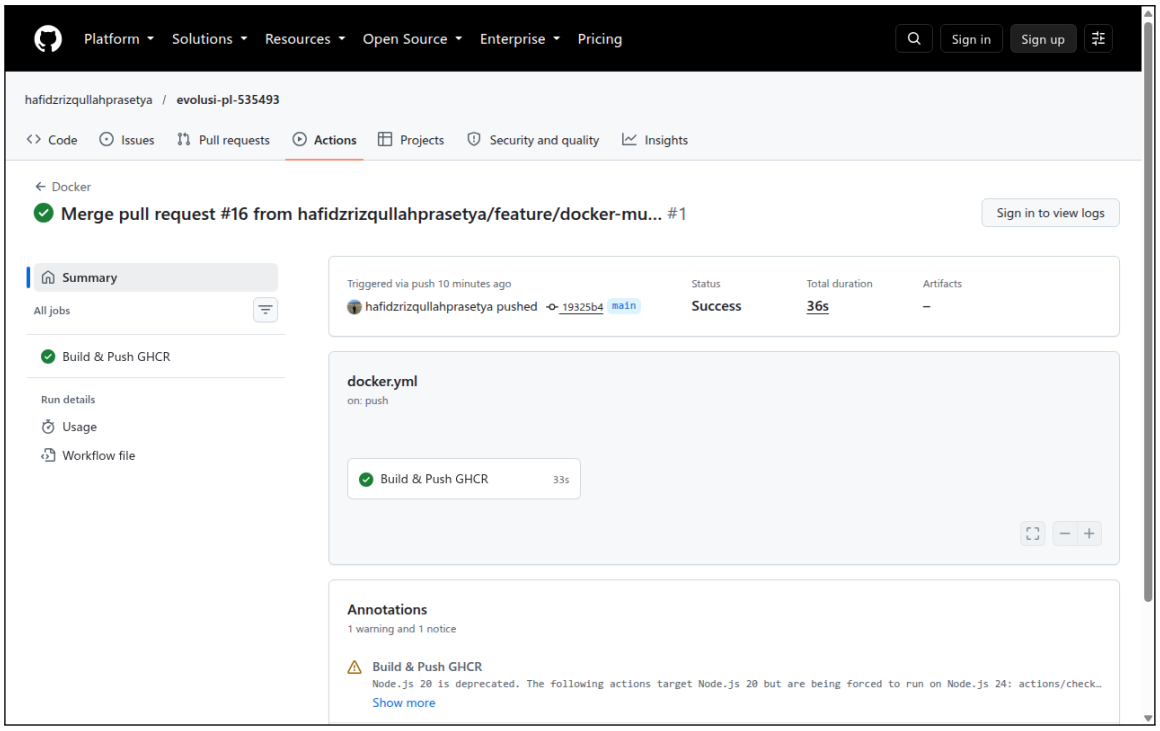
Gambar 4: Pengujian pemanggilan API `/api/tugas` dari luar container menghasilkan status HTTP 200 OK beserta payload JSON.

Respon yang diterima menunjukkan kode `HTTP/1.1 200 OK` dengan header

`Content-Type: application/json`. Data JSON yang memuat daftar entitas tugas berhasil diambil dari basis data SQLite di dalam container tanpa hambatan izin baca-tulis.

5.5 Bukti Eksekusi Workflow GitHub Actions

Setelah Pull Request nomor 16 digabungkan ke cabang `main`, workflow `Docker` secara otomatis dijalankan oleh runner GitHub Actions. Gambar 5 memperlihatkan log eksekusi workflow yang berhasil diselesaikan dengan status hijau:

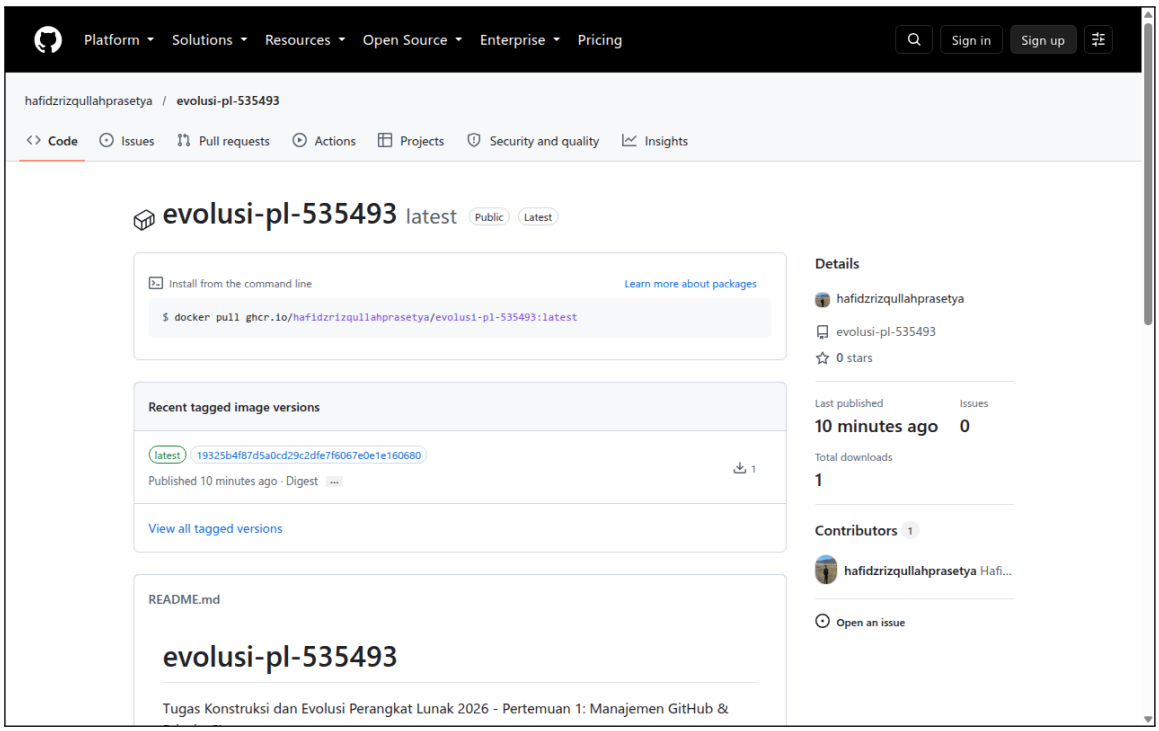


Gambar 5: Tampilan antarmuka GitHub Actions membuktikan eksekusi workflow `Docker` berjalan sukses (hijau) dalam 33 detik.

Job `Build & Push GHCR` sukses menyelesaikan seluruh tahapan: mulai dari persiapan variabel repositori, autentikasi ke registri `ghcr.io`, build image multi-stage, hingga proses push dua tag image ke GitHub Container Registry.

5.6 Bukti Publikasi Package di GitHub Container Registry

Hasil akhir publikasi dapat dilihat langsung pada tab Packages di repositori GitHub `hafidzrizqullahprasetya/evolusi-pl-535493`:



Gambar 6: Halaman package `evolusi-pl-535493` di GitHub Container Registry dengan visibilitas Public.

Pada halaman tersebut terlihat rincian paket container:

- Nama paket: `evolusi-pl-535493`.
- Status visibilitas: **Public** (dapat diunduh secara bebas oleh publik tanpa autentikasi).
- Tag rilis aktif: `latest` dan `19325b4f87d5a0cd29c2dfe7f6067e0e1e160680` (sesuai nomor commit push pada cabang main).
- Perintah instalasi publik:

```
docker pull ghcr.io/hafidzrizqullahprasetya/evolusi-pl-535493:latest

```

Kesimpulan

Berdasarkan serangkaian langkah praktikum dan pengujian yang telah dilaksanakan pada pertemuan kelima ini, diperoleh kesimpulan sebagai berikut:

1. Penerapan arsitektur *multi-stage build* berhasil memisahkan tahap persiapan dependensi Composer dari tahap runtime produksi, sehingga berkas sampah kompilasi tidak mengotori lingkungan rilis.

2. Penggunaan base image minimal berbasis `php:8.4.26-cli-alpine` berhasil memangkas ukuran container image secara drastis dari **940 MB** menjadi **266 MB**, menghasilkan penurunan ukuran sebesar **674 MB** atau setara dengan efisiensi **71,70%**.

3. Aspek keamanan container meningkat dengan diterapkannya instruksi `USER www-data`, memastikan container tidak beroperasi dengan privilese akun *root*.

4. Mekanisme `HEALTHCHECK` berhasil mengawasi keandalan layanan web secara otomatis di dalam container, yang dibuktikan dengan status `(healthy)` pada pemantauan `docker ps`.

5. Jalur CI/CD untuk pengiriman artefak container berhasil diotomatisasi melalui GitHub Actions, menghasilkan publikasi image ke GitHub Container Registry (GHCR) dengan tag penanda commit SHA serta visibilitas publik yang dapat diakses secara terbuka.

Tautan Pengumpulan Praktikum:

- Repositori Proyek: `https://github.com/hafidzrizqullahprasetya/evolusi-pl-535493`
- Halaman Package GHCR: `https://github.com/hafidzrizqullahprasetya/evolusi-pl-535493/pkgs/container/evolusi-pl-535493`

Daftar Pustaka

[1] Docker Inc., "Docker overview." [Online]. Tersedia: https://docs.docker.com/get-started/overview/.

[2] Docker Inc., "Multi-stage builds." [Online]. Tersedia: https://docs.docker.com/build/building/multi-stage/.

[3] Docker Inc., "Docker security and best practices." [Online]. Tersedia: https://docs.docker.com/engine/security/.

[4] Docker Inc., "HEALTHCHECK directive reference." [Online]. Tersedia: https://docs.docker.com/reference/dockerfile/#healthcheck.

[5] GitHub Docs, "Working with the Container registry." [Online]. Tersedia: https://docs.github.com/en/packages/working-with-a-github-packages-registry/working-with-the-container-registry.

[6] Laravel LLC, "Deployment - Laravel 12.x Documentation." [Online]. Tersedia: https://laravel.com/docs/deployment.